

Hướng Dẫn Xây Dựng Lại DigitalAsset Guard AI Copilot — Từ Đầu

Đi kèm với `SPEC.md` (bản đặc tả duy nhất). Mỗi phần dưới đây = 1 buổi làm việc, làm xong phần nào kiểm tra xong phần đó rồi mới sang phần tiếp theo. Đừng nhảy cóc.

Phần 0 — Chuẩn bị môi trường

Mục tiêu: có một khởi điểm sạch, không lẫn code cũ vào code mới.

Việc cần làm:

- Đổi tên thư mục repo hiện tại: `digitalasset_guard` → `digitalasset_guard_v1_archive`. Không sửa gì trong đó nữa, chỉ để tham khảo/copy khi cần.
- Tạo thư mục mới `digitalasset_guard/`, `cd` vào đó, `git init`.
- Copy `SPEC.md` vào gốc thư mục mới. Đây là file bạn sẽ mở ra đọc trước khi làm bất kỳ phần nào bên dưới.
- Tạo virtual environment: `python -m venv venv` → activate.
- Tạo `requirements.txt` khởi điểm (sẽ bổ sung dần theo từng phần):

```
python-dotenv  
pydantic
```

- Tạo `.env` (rỗng, sẽ điền dần) và `.gitignore` (thêm `venv/`, `.env`, `*.pkl`, `data/raw/`, `__pycache__`).
- Dựng khung thư mục rỗng đúng như mục 6 trong `SPEC.md` (dùng `mkdir -p` cho toàn bộ cây thư mục, thêm file `.gitkeep` vào các thư mục rỗng).

Lưu ý:

- Đừng copy bất kỳ file `.py` nào từ v1 vào lúc này. Bạn sẽ chỉ copy từng đoạn code cụ thể khi làm tới phần tương ứng, sau khi đã hiểu `SPEC.md` yêu cầu gì — tránh mang theo lỗi cũ mà không biết.
- Không cài đặt Neo4j/ChromaDB ngay bây giờ. Để dành đến Phần 3 và Phần 6.

Hoàn thành khi: có cấu trúc thư mục rỗng đúng `SPEC.md`, `venv` chạy được, `git` đã init.

Phần 1 — Nền tảng: `core/config.py` + `core/state.py`

Mục tiêu: dựng "khung xương" mà mọi module sau này đều phụ thuộc vào. Làm sai ở đây sẽ phải sửa lại toàn bộ agent sau này, nên làm cẩn thận nhất có thể ngay từ đầu.

Việc cần làm:

1. `core/config.py`:

- Đọc biến môi trường từ `.env` (dùng `python-dotenv`).
- Khai báo các hằng số: ngưỡng báo cáo (500 triệu VND / 1.000 USD), đường dẫn `data/`, `models/`, `reports/output/`.
- Khai báo cờ `DEMO_MODE = True/False` — khi `True`, hệ thống dùng NetworkX + dữ liệu mock thay vì Neo4j/API thật (để bạn chạy được ngay cả khi chưa cài Neo4j).

2. `core/state.py`:

- Định nghĩa `AMLState` (dùng `TypedDict` hoặc Pydantic model) — đây là "hộp dữ liệu" đi xuyên suốt qua tất cả agent trong LangGraph.
- **Chỉ khai báo các trường sau (đúng SPEC.md mục 4), không thêm trường PII gốc nào:**

python

```
class AMLState(TypedDict):
    tx_hash: str
    wallet_from: str
    wallet_to: str
    amount_vnd: float
    hashed_fullname: str | None
    hashed_id_number: str | None
    hashed_account_number: str | None
    risk_score_classifier: float | None
    kyc_flags: list | None
    graph_risk_score: float | None
    legal_citations: list | None
    final_risk_score: float | None
    report_path: str | None
    approval_status: str | None # "pending" | "approved" | "rejected"
```

- Viết docstring ngay đầu file nhắc lại quy tắc: *"Không bao giờ thêm trường chứa PII gốc (fullname, id_number, account_number) vào state này. Chỉ dùng bản đã băm."*

Lưu ý quan trọng:

- Đây là lúc dễ bị cám dỗ "thêm tạm vài trường cho tiện" nhất — đừng làm. Mọi trường thêm vào `AMLState` sau này phải quay lại đối chiếu với SPEC.md trước.
- Nếu dùng Pydantic thay TypedDict, dùng `model_config = {"extra": "forbid"}` để không lỡ tay thêm trường ngoài ý muốn ở agent sau này.

Cách kiểm tra: viết 1 file test nhỏ `tests/test_state.py`, thử tạo 1 instance `AMLState` với dữ liệu mẫu, in ra, xác nhận không có lỗi type.

Hoàn thành khi: `config.py` load được `.env`, `state.py` tạo được object mẫu không lỗi.

Phần 2 — Privacy Layer (SHA-256 Masking)

Mục tiêu: dựng ranh giới bảo mật bắt buộc phải có trước khi bất kỳ agent nào chạm vào dữ liệu khách hàng.

Việc cần làm:

1. Tạo `core/privacy_layer.py` với hàm chính:

python

```
def mask_pii(raw_customer: dict, salt: str) -> dict:
    """
    Input: {"fullname": ..., "id_number": ..., "account_number": ...}
    Output: {"hashed_fullname": ..., "hashed_id_number": ..., "hashed_account": ...}
    """
```

2. Dùng `hashlib.sha256((value + salt).encode()).hexdigest()` cho từng trường.
3. Đọc `salt` từ `.env` (biến `PII_SALT`), sinh ngẫu nhiên 1 lần và lưu cố định — không hard-code trong source.
4. Viết thêm 1 hàm `assert_no_raw_pii(state: dict)` — quét state, nếu phát hiện key nào nằm trong danh sách cấm (`fullname`, `id_number`, `account_number` — không có tiền tố `hashed_`) thì raise Exception ngay. Hàm này sẽ được gọi ở đầu mỗi agent từ Phần 4 trở đi, như một "chốt kiểm tra" tự động.

Lưu ý quan trọng:

- Salt cố định trong `.env` cho MVP (đã chốt ở SPEC.md mục 8) — không cần xoay vòng theo phiên, nhưng phải ghi rõ giới hạn này vào phần "Hạn chế" của báo cáo cuối cùng khi bảo vệ.
- Không log giá trị PII gốc ra console/file log ở bất kỳ đâu trong hàm `mask_pii`, kể cả khi debug — xóa print/log ngay sau khi test xong.

Cách kiểm tra: viết `tests/test_privacy_layer.py`:

- Test 1: cùng input → cùng hash (deterministic).
- Test 2: 2 input khác nhau → 2 hash khác nhau.
- Test 3: gọi `assert_no_raw_pii` với 1 dict có key `fullname` → phải raise lỗi.
- Test 4: gọi với dict chỉ có `hashed_fullname` → không lỗi.

Hoàn thành khi: cả 4 test trên pass.

Phần 3 — Chuẩn bị dữ liệu

Mục tiêu: có đủ dữ liệu thật để agent ở các phần sau xử lý, không cần đụng tới model/logic AI ở phần này.

Việc cần làm:

1. Elliptic Dataset ((data/raw/elliptic/)):

- Tải bộ (Elliptic Bitcoin Dataset) (features, classes, edgelist) — 3 file CSV chuẩn của bộ dữ liệu gốc.
- Viết (scripts/03_load_elliptic.py): đọc 3 file CSV, merge lại thành 1 DataFrame có cột (time_step), (class) (1=illicit, 2=licit, unknown=bỏ), và các cột đặc trưng.

2. OFAC SDN ((data/raw/ofac/sdn.xml)):

- Tải file XML danh sách SDN từ trang OFAC (định dạng XML, **không phải .txt** — đã chốt ở SPEC.md).
- Viết (scripts/01_check_ofac.py): dùng (xml.etree.ElementTree.iterparse) để đọc từng entry mà không load toàn bộ file vào RAM, trích ra danh sách tên + địa chỉ ví (nếu có) → lưu gọn thành (data/processed/sample_ofac_wallet.txt) để agent KYC dùng nhanh mà không phải parse lại XML mỗi lần chạy.

3. Etherscan ((data/raw/etherscan/)):

- Đăng ký free API key Etherscan, lưu vào (.env) (ETHERSCAN_API_KEY).
- Viết (scripts/02_fetch_etherscan_sample.py): gọi API lấy lịch sử giao dịch của 1 nhóm ví mẫu (có thể chọn vài ví công khai đã biết dính rủi ro để demo thuyết phục hơn), lưu ra JSON/CSV thô.
- (Quyết định BscScan: theo khuyến nghị SPEC.md mục 8 — chỉ làm Ethereum trong MVP, bỏ qua BscScan, ghi chú vào README là hướng mở rộng sau.)

4. Văn bản pháp luật ((data/legal_docs/)):

- (thong_tu_27_2025.txt) — nội dung Thông tư 27/2025/TT-NHNN (ít nhất các điều khoản về ngưỡng báo cáo, thời hạn STR, human-in-the-loop).
- (thong_tu_32_2026.txt) — nội dung Thông tư 32/2026/TT-BTC (thuế TNCN 0.1%) — **bắt buộc phải có**, đây là lỗ hổng đã phát hiện ở lượt rà soát trước.
- (fatf_recommendations.txt) — tóm tắt khuyến nghị FATF liên quan (tùy chọn, làm phong phú RAG).

5. Mock banking data ((data/mock/)):

- (scripts) hoặc file trực tiếp sinh (customers.json): danh sách khách hàng giả lập (tên, CCCD, số tài khoản, ví liên kết) — dữ liệu này sẽ đi qua Privacy Layer ở Phần 4 trước khi vào agent.

Lưu ý quan trọng:

- Ở phần này **chưa cần gọi Privacy Layer** — đây chỉ là bước thu thập dữ liệu thô. Việc băm PII sẽ xảy ra ngay tại điểm nhận webhook trong `(demo_run.py)` (Phần 10), không phải ở đây.
- Giữ file dữ liệu thô trong `(.gitignore)` (đừng commit file Elliptic/OFAC lên git).

Cách kiểm tra: chạy từng script (`(00_healthcheck.py)` kiểm tra kết nối API/file tồn tại, rồi `(01)`, `(02)`, `(03)`), xác nhận in ra số dòng/số bản ghi đọc được > 0, không lỗi.

Hoàn thành khi: cả 4 nguồn dữ liệu (Elliptic, OFAC XML, Etherscan, luật) đã có sẵn trên đĩa và đọc được.

Phần 4 — Transaction Assistant + Huấn luyện mô hình

Mục tiêu: có 1 agent độc lập chấm điểm rủi ro sơ bộ, đánh giá đúng bằng chỉ số phù hợp với dữ liệu mất cân bằng.

Việc cần làm:

1. `(agents/train_classifier.py)`:
 - Load DataFrame Elliptic từ Phần 3.
 - **Chia tập theo `(time_step)`:** `(time_step <= 34)` → train, `(> 34)` → test. Không dùng `(train_test_split)` ngẫu nhiên.
 - Loại bỏ nhãn "unknown".
 - Huấn luyện `(XGBClassifier)` với `(scale_pos_weight = (y_train==0).sum() / (y_train==1).sum())`.
 - Lưu model ra `(models/xgboost_aml.pkl)`.
2. `(tests/evaluate_model.py)`:
 - Tính Accuracy, Precision, Recall, F1 (dùng `(precision_recall_fscore_support, average="binary")`).
 - Tính AUC-PR bằng `(average_precision_score)` — **không dùng AUC-ROC làm chỉ số chính**.
 - In bảng kết quả ra console đúng format SPEC.md mục 3.1, để bạn copy thẳng vào báo cáo khi điền số liệu thật.
3. `(agents/transaction_classifier.py)`:
 - Load model đã huấn luyện.
 - Hàm nhận vào `(AMLState)` (đã qua Privacy Layer), trích đặc trưng giao dịch thô (giá trị VND quy đổi, phí gas, tần suất), trả về `(risk_score_classifier)`.
 - Gọi `(assert_no_raw_pii(state))` ngay đầu hàm.

Lưu ý quan trọng:

- Đừng vội hài lòng nếu Accuracy cao — với tỷ lệ illicit ~2%, một model dự đoán "tất cả hợp lệ" vẫn đạt ~98% Accuracy nhưng vô dụng. Nhìn vào Recall và AUC-PR trước.
- Ghi lại con số thật (không phải ước lượng) vào báo cáo — hội đồng chắc chắn sẽ hỏi.

Cách kiểm tra: chạy `python -m agents.train_classifier` rồi `python -m tests.evaluate_model`, xác nhận in ra bảng số liệu hợp lý (Recall không được = 0, AUC-PR nên > tỷ lệ nền 2%).

Hoàn thành khi: có model đã lưu, bảng đánh giá in ra đầy đủ 5 chỉ số, `transaction_classifier.py` chạy độc lập trả về điểm số hợp lệ.

Phần 5 — KYC Assistant

Mục tiêu: agent độc lập sàng lọc địa chỉ ví/tên qua danh sách trùng phạt.

Việc cần làm:

1. `agents/kyc_verification.py`:
 - Load `data/processed/sample_ofac_wallet.txt` (đã xử lý sẵn từ Phần 3, không parse lại XML mỗi lần).
 - Hàm nhận `AMLState`, so khớp `wallet_from`/`wallet_to` và `hashed_fullname` (chỉ so khớp trên bản đã băm hoặc trên các trường không nhạy cảm như địa chỉ ví) với danh sách đen.
 - Dùng thuật toán so khớp mờ (Levenshtein Distance — thư viện `python-Levenshtein` hoặc `rapidfuzz`) để bắt các biến thể tên gần đúng.
 - Trả về `kyc_flags`: danh sách cờ cảnh báo (ví dụ `[["wallet_match_ofac", "name_similarity_92%"]]`).
 - Gọi `assert_no_raw_pii(state)` ngay đầu hàm.

Lưu ý quan trọng:

- Vì tên khách hàng đã bị băm ở Privacy Layer, việc so khớp mờ theo tên **chỉ khả thi nếu bạn so khớp trước khi băm** (tức so khớp xảy ra ngay trong hoặc trước Privacy Layer, agent này chỉ nhận kết quả cờ đã có sẵn) — hoặc **chỉ so khớp theo địa chỉ ví** (không cần biết tên) ở tầng agent này. Quyết định rõ 1 trong 2 cách và ghi chú lại trong code, đừng để mơ hồ — đây là điểm dễ gây lỗi logic nhất trong toàn bộ agent.
 - *Khuyến nghị:* để KYC Assistant chỉ so khớp theo địa chỉ ví (không PII), còn việc so khớp tên diễn ra như một bước riêng **trong** Privacy Layer/webhook trước khi băm, kết quả trả ra dạng cờ boolean đính kèm vào state.

Cách kiểm tra: chạy thử với 1 địa chỉ ví có trong danh sách đen mẫu (tự chèn 1 dòng test vào `sample_ofac_wallet.txt`) → xác nhận trả về cờ đúng; chạy với ví sạch → không có cờ.

Hoàn thành khi: agent chạy độc lập, phân biệt đúng ví có/không nằm trong danh sách đen.

Phần 6 — Graph Assistant

Mục tiêu: agent phân tích cấu trúc dòng tiền đa hop.

Việc cần làm:

1. Cài Neo4j Community Edition local (hoặc dùng chế độ `DEMO_MODE=True` với NetworkX nếu chưa muốn cài Neo4j ngay).
2. `db/neo4j_setup.py`: script tạo schema/constraint cơ bản (node `wallet`, relationship `TRANSFER`).
3. `agents/graph_aml.py`:
 - Nếu `DEMO_MODE=False`: chạy Cypher query trên Neo4j.
 - Nếu `DEMO_MODE=True`: dựng đồ thị tạm bằng NetworkX từ dữ liệu Etherscan đã crawl (Phần 3).
 - Cài đặt PPR (dùng `networkx.pagerank` cho demo, hoặc Neo4j GDS `gds.pageRank` cho production) với vector cá nhân hóa tập trung vào ví đen đã biết từ KYC Assistant.
 - Cài đặt Louvain (`networkx.algorithms.community.louvain_communities`) cho demo, hoặc Neo4j GDS Louvain cho production).
 - Trả về `graph_risk_score` (dựa trên PPR) và thông tin cộng đồng (dùng để phát hiện đảo gian lận).
 - Gọi `assert_no_raw_pii(state)` ngay đầu hàm.

Lưu ý quan trọng:

- Đây là agent phức tạp kỹ thuật nhất — làm việc trên dữ liệu ví thật (không nhạy cảm PII vì chỉ là địa chỉ công khai on-chain), nên **không cần** Privacy Layer áp dụng lên địa chỉ ví, chỉ áp dụng lên thông tin định danh khách hàng ngoài đời thực.
- Nếu chạy demo với NetworkX, số lượng node nên giới hạn (vài trăm đến vài nghìn ví) để chạy mượt trên máy cá nhân — không cần load toàn bộ Etherscan.

Cách kiểm tra: chạy thử với 1 ví có kết nối gần (1-2 hop) tới ví đen đã biết →

`graph_risk_score` phải cao hơn rõ rệt so với ví không có kết nối nào.

Hoàn thành khi: agent chạy độc lập, phân biệt được ví "gần" ví đen và ví "xa".

Phần 7 — RAG Assistant

Mục tiêu: agent tra cứu và trích dẫn đúng căn cứ pháp lý.

Việc cần làm:

1. `db/vector_db.py`: khởi tạo ChromaDB local, load và embed nội dung từ `data/legal_docs/` (cả `thong_tu_27_2025.txt` và `thong_tu_32_2026.txt`).

2. `agents/regulation_rag.py`:

- Nhận đặc trưng cấu trúc từ Graph Assistant (`graph_risk_score`, community ID) + `risk_score_classifier`.
- Mã hóa các đặc trưng này thành câu ngữ cảnh (ví dụ: "Giao dịch có PPR score cao, thuộc cộng đồng nghi vấn, giá trị vượt ngưỡng 500 triệu VND").
- Truy vấn ChromaDB, lấy đoạn luật liên quan nhất.
- Nếu giao dịch có yếu tố tài sản số phát sinh thu nhập → truy vấn thêm Thông tư 32 để trích dẫn nghĩa vụ thuế 0.1%.
- Gọi API LLM (OpenAI/Gemini) để tổng hợp thành đoạn văn bản lập luận pháp lý ngắn gọn, có trích dẫn.
- Trả về `legal_citations`.

Lưu ý quan trọng:

- Đảm bảo cả 2 nguồn luật (27 và 32) đều được embed — kiểm tra bằng cách thử 1 câu hỏi chỉ liên quan tới thuế, xác nhận RAG trả về đúng đoạn từ Thông tư 32 chứ không chỉ luôn trả Thông tư 27.

Cách kiểm tra: thử 2 query mẫu — 1 câu về ngưỡng báo cáo (phải trả Thông tư 27), 1 câu về thuế tài sản số (phải trả Thông tư 32).

Hoàn thành khi: agent phân biệt đúng nguồn luật theo ngữ cảnh truy vấn.

Phần 8 — Report Assistant

Mục tiêu: tổng hợp toàn bộ kết quả thành bản dự thảo STR.

Việc cần làm:

1. `agents/alert_report.py`:

- Tính `final_risk_score = 0.2*risk_score_classifier + 0.3*len(kyc_flags biến đổi thành số) + 0.5*graph_risk_score` (chuẩn hóa các thành phần về cùng thang điểm 0-1 trước khi cộng — đây là chi tiết kỹ thuật cần làm cẩn thận, ghi rõ công thức chuẩn hóa trong docstring).
- Docstring giải thích lý do chọn trọng số 0.2/0.3/0.5 (đã có sẵn nội dung trong SPEC.md mục 3.5, copy vào).
- Nếu `final_risk_score >= 0.7` (ngưỡng có thể điều chỉnh, ghi rõ lý do chọn ngưỡng này): dùng `python-docx` điền vào template `reports/templates/` (Mẫu số 04 theo Thông tư 27), đính kèm mô tả sơ đồ dòng tiền (có thể render ảnh từ Graph Assistant nếu có thời gian, hoặc mô tả bằng text danh sách các hop).
- Lưu file ra `reports/output/STR_REPORT_<tx_hash>.docx`.
- Set `approval_status = "pending"`.

Lưu ý quan trọng:

- Đây là nơi duy nhất trong toàn hệ thống được phép "giải mã ngược" để hiển thị tên khách hàng cho chuyên viên xem trong báo cáo cuối — nhưng phải tra cứu qua 1 bảng ánh xạ (hash → tên gốc) được lưu riêng, bảo vệ nghiêm ngặt (không nằm chung với AMLState), không phải giải mã hash SHA-256 (không thể giải mã 1 chiều) mà là tra bảng lookup được lưu an toàn tại thời điểm Privacy Layer băm dữ liệu.

Cách kiểm tra: chạy thử với 1 state mẫu có (final_risk_score) cao → xác nhận file (.docx) được tạo ra, mở lên đọc được nội dung đầy đủ.

Hoàn thành khi: file STR mẫu được sinh ra đúng định dạng, đọc được, có đủ thông tin.

Phần 9 — Ghép nối LangGraph (core/graph_builder.py)

Mục tiêu: chỉ làm phần này khi cả 5 agent + Privacy Layer đã chạy độc lập ổn (Phần 2, 4-8).

Việc cần làm:

1. Định nghĩa graph LangGraph với các node: (privacy_layer → transaction_classifier → kyc_verification → graph_aml (2 nhánh song song có thể) → regulation_rag → alert_report → interrupt (chờ duyệt)).
2. Cấu hình (interrupt) tại node sau (alert_report) — dừng lại, trả state hiện tại ra ngoài để UI hiển thị.
3. Viết hàm (resume_after_approval(state, decision)) xử lý khi chuyên viên Approve/Reject.
4. Có fallback (run_simple_pipeline()) (gọi tuần tự các hàm Python thường) cho trường hợp không cài LangGraph — theo đúng tinh thần v1 cũ, giữ lại pattern này.

Lưu ý quan trọng:

- Đừng sửa logic bên trong bất kỳ agent nào ở bước này — nếu graph chạy sai, khả năng cao là do cách nối node/cạnh, không phải do agent (agent đã test độc lập ở các phần trước).

Cách kiểm tra: chạy toàn bộ pipeline với 1 giao dịch mẫu từ đầu đến điểm interrupt, xác nhận dừng đúng chỗ và state đầy đủ dữ liệu từ tất cả agent.

Hoàn thành khi: pipeline đầy đủ chạy được từ đầu đến điểm chờ duyệt, không lỗi.

Phần 10 — API Gateway + UI + Demo Script

Mục tiêu: lớp trình diễn cuối cùng, làm sau khi mọi thứ phía dưới đã ổn định.

Việc cần làm:

1. (api/main.py) (FastAPI):

- 1 endpoint (`POST /screen-wallet`) — nhận địa chỉ ví, trả về kết quả sàng lọc OFAC + risk score sơ bộ (dùng lại `kyc_verification.py` + `transaction_classifier.py`, không cần chạy full LangGraph). Đây là demo cho gói API-as-a-Service.

2. `ui/app.py` (Streamlit):

- Form nhập giao dịch → gọi `demo_run.py` → hiển thị kết quả từng agent theo thời gian thực → hiển thị điểm dừng chờ duyệt với nút Approve/Reject → hiển thị link tải file `.docx` khi Approve.

3. `demo_run.py`: script chạy full workflow từ webhook giả lập đến STR, dùng để demo trực tiếp khi thi/báo cáo.

Lưu ý quan trọng:

- UI không cần đẹp, chỉ cần thể hiện đúng luồng nghiệp vụ 8 bước trong SPEC.md mục 2 — hội đồng chăm ý tưởng/kỹ thuật, không chăm giao diện.

Cách kiểm tra: chạy `streamlit run ui/app.py`, thử toàn bộ luồng từ nhập giao dịch đến tải file STR.

Hoàn thành khi: demo chạy mượt từ đầu đến cuối không cần can thiệp thủ công vào code.

Phần 11 — Kiểm thử tổng thể & hoàn thiện báo cáo

Việc cần làm:

1. Chạy lại toàn bộ `tests/` một lượt, đảm bảo pass hết.
2. Điền số liệu thật (Recall, F1, AUC-PR) từ Phần 4 vào báo cáo khả thi (thay chỗ `__%` trong bảng).
3. Copy công thức Weighted Risk Score + lý do chọn trọng số từ code vào báo cáo (mục 6/7).
4. Rà lại 1 lượt: mọi thứ trong `SPEC.md` mục "Việc cần bạn xác nhận" (BscScan, salt) đã được quyết định và ghi rõ trong README/báo cáo.
5. Chuẩn bị sẵn câu trả lời cho câu hỏi dễ bị hỏi nhất: *"Recall bao nhiêu, tại sao ưu tiên Recall/AUC-PR hơn Accuracy?"*

Hoàn thành khi: bạn có thể chạy demo trực tiếp trước hội đồng mà không lo lỗi, và trả lời được câu hỏi kỹ thuật dựa trên số liệu thật.

Nguyên tắc xuyên suốt khi làm từng phần với AI

Ở mỗi phần, khi nhờ AI viết code, dùng đúng mẫu câu:

"Đây là SPEC.md. Đã build xong đến Phần [X]. Viết ĐỌC LẬP module của Phần [Y] theo đúng mục [số mục] trong SPEC.md. Không sửa file cũ. Nếu cần input từ phần chưa làm, dùng mock data và ghi TODO."

Không gộp 2 phần vào 1 lệnh, kể cả khi thấy "có vẻ đơn giản, làm luôn cho nhanh" — chính suy nghĩ đó là nguyên nhân gây mất định hướng ở bản v1.